

Ревью кода на Python

Правила для успешного прохождения code review

Соблюдайте [стандарт](#).

Пишите [документацию](#).

Основные цели code review

- Приведение кода к единому стилю кодирования PEP-8, для более простого чтения кода.
- Снижение цикломатической сложности кода, для повышения понимания логики кода.
- Проверка кода на использование рискованных или непродуктивных шаблонов кода (анти-паттерн).
- Анализ нарушений SOLID в архитектуре проекта (согласно ГОСТ Р 57100-2016/ISO/IEC/IEEE 42010:2011) для повышения гибкости и расширяемости кода.

Критерии review

Статический анализ Pylint

Анализ кода утилитой pylint ([настройки](#)) для поисков дублирования кода, соблюдения стандарта PEP-8 и поиска анти-паттернов.

Успешной оценкой pylint считается 8 баллов (или выше) из 10.

Статический анализ Radon

Анализ кода утилитой radon для оценки цикломатической сложности кода. Цикломатическая сложность делится утилитой по категориям A, B, C, D, E и F.

- A (сложность до 5) — как правило, функция хелпер.
- B (сложность до 10) — являются хорошими для чтения и понимания блоком
- C (до 20) — блок умеренной сложности
- D (до 30) — сложный блок
- E и F (от 30) — очень сложные и не стабильные блоки. По статистике, с большой вероятностью, содержащие скрытые ошибки.

Хорошим кодом является тот, где блоков категории D менее 20% от кода со сложностью B и C.

Код, где есть код со сложностью E и F является опасным, плохо расширяемым и тяжело поддерживаемым. Такие блоки необходимо переписывать не задумываясь.

Проверка кода на анти-паттерны

Есть ряд шаблонов, которые часто используют в проектах. Такие шаблоны говорят о плохом разделении кода или имеют большой риск приводить к ошибкам, которые тяжело обнаружить.

- Божественный объект.
- Использования статичных свойств класса для инициализации значений.
- Одиночка.

И так [далее](#).

Анализ нарушения SOLID

SOLID — это математически выведенные, принципы ООП. Их нарушение приводит к малой гибкости кода и проблематичному расширению.

Использование ORX только как точки входа

Проверка на то, что методы, описанные в огах файлах, являются лишь точками входа, и в них нет прикладной логики. Связно с проблемами diff сравнения. Редактированием и поиском кода.

Не использование общих имен в python модулях

В связи с тем, что при загрузке модулей они попадают в общую область видимости, то использование общих имен в модулях, находящихся в корне СБИС-модулей, приводит к тому, что происходит конфликт имен. И тот модуль, который будет загружен первый и будет отдаваться всем.

Рекомендации по принципам разработки в СБИС

Рекомендуется платформенные БЛ-методы использовать лишь как точки входа в приложение.

А всю логику описывать в классах с простым состоянием (в python модулях). И в рамках одного БЛ-вызова не обращаться к своей же логике через ORX API (далее stateless), а вызывать логику через обращение к классам (далее stateful).

Связано это с тем, что методы в stateless приложении общаются с помощью идентификаторов.

И для получения одних и тех же данных повторно запрашивают их из БД. А в наших реалиях, зачастую, создаются вложенные транзакции в рамках одного БЛ-вызова.

Таким образом, переходя на stateful модель в рамках запроса, мы получим выигрыш в ресурсах и более поддерживаемый код.